KỸ THUẬT LẬP TRÌNH / JAVA

CHƯƠNG 1: INTRODUCTION TO JAVA PROGRAMMING

Buổi học tích hợp Lý thuyết & Thực hành tại lớp (Thời lượng: 3 Tiết)

Giảng viên Học phần
Khoa Công nghệ Thông tin

Dành cho sinh viên Khóa 2026 – Lớp IT26M

Dựa trên giáo trình: *Building Java Programs* (Stuart Reges & Marty Stepp)

Năm học 2026

Mục tiêu học tập bài học (Course Learning Outcomes – CLOs)

Chuẩn đầu ra sau khi hoàn thành Chương 1 (Buổi học 3 tiết):

- ✓ **CLO 1.1 (Hiểu):** Giải thích bản chất kiến trúc máy tính, hệ nhị phân, cơ chế biên dịch vs. thông dịch, vai trò của **máy ảo JVM** và mã **Bytecode** (WORA).
- ✓ **CLO 1.2 (Áp dụng):** Viết thành thạo khung chương trình Java cơ bản (`class`, `main`), phân biệt `print` và `println`, sử dụng chính xác các **ký tự thoát** (`\n, \t, \", \\`).
- ✓ **CLO 1.3 (Phân tích):** Nhận diện và phân loại chính xác **3 loại lỗi** (Syntax, Runtime, Logic); đọc thông báo lỗi từ trình biên dịch để gỡ lỗi (debug).
- ✓ **CLO 1.4 (Thiết kế):** Vận dụng tư duy **Phân rã thủ tục (Procedural Decomposition)** và nguyên lý **DRY**; khai báo và gọi phương thức tĩnh (`static methods`); phân tích luồng điều khiển (*Flow of Control*).
- ✓ **CLO 1.5 (Sáng tạo):** Độc lập viết, biên dịch qua CLI (`javac/java`) và IDE một chương trình hoàn chỉnh vẽ hình nghệ thuật ASCII có cấu trúc nhiều cấp.

Lộ trình buổi học 3 tiết (Tích hợp Lý thuyết & Thực hành)

TIẾT 1 (50 phút)

Nền tảng & Khởi đầu:

- Khái niệm tính toán & Nhị phân
- Compiler vs. Interpreter
- Kiến trúc JVM & Bytecode
- Quy trình: Edit-Compile-Run
- Chương trình `Hello.java`
- Lệnh `print` vs `println`

[Lab 1]: Viết & chạy Java trên CLI và IDE.

TIẾT 2 (50 phút)

Cú pháp & Gỡ lỗi:

- Ký tự thoát (*Escape Seq*)
- Định danh, Từ khóa, CamelCase
- Quy ước chú thích (Comments)
- Phân loại 3 nhóm lỗi:
  - Lỗi cú pháp (Syntax)
  - Lỗi thực thi (Runtime)
  - Lỗi ngữ nghĩa (Logic)
- Chiến lược đọc lỗi & gỡ lỗi

[Lab 2]: Thử thách Debug & Escape Strings.

TIẾT 3 (50 phút)

Phân rã & Case Study:

- Tư duy phân rã Top-down
- Phương thức tĩnh (`static`)
- Luồng điều khiển (*Control Flow*)
- Mục tiêu: Structure & DRY
- Case Study `DrawFigures`:
  - Bản 1: Naïve (đơn khối)
  - Bản 2: Phân rã cấp 1
  - Bản 3: Phân rã triệt để

TIẾT 1

NỀN TẢNG HỆ THỐNG TÍNH TOÁN, KIẾN TRÚC MÁY ẢO JVM & XÂY DỰNG CHƯƠNG TRÌNH JAVA ĐẦU TIÊN

Thời lượng: 50 phút | 35’ Lý thuyết tương tác + 15’ Hands-on Lab 1

1.1 Tại sao chúng ta cần học lập trình? (Why Programming?)

Bản chất của khoa học máy tính

> *"Người ta thường nói rằng một người không thực sự hiểu điều gì đó cho đến khi họ dạy nó cho người khác. Trên thực tế, một người **không thực sự hiểu** điều gì đó cho đến khi họ **dạy nó cho một chiếc máy tính**, tức là diễn đạt nó dưới dạng một thuật toán."*
>
> *— Donald Knuth (Turing Award, Cha đẻ ngành Phân tích thuật toán)*

- **Máy tính:** Thiết bị phần cứng vạn năng (General-purpose device).
- **Chương trình (Program):** Tập hợp các chỉ thị ra lệnh cho phần cứng.
- **Lập trình:** Rèn luyện tư duy giải quyết vấn đề có

Thành phần cốt lõi của máy tính

- **CPU (Bộ vi xử lý trung tâm):** "Bộ não" thực thi các phép tính logic và số học.
- **Bộ nhớ chính (RAM):** Lưu trữ tạm thời, tốc độ cực nhanh, mất dữ liệu khi mất nguồn.
- **Bộ nhớ thứ cấp (Ổ cứng/SSD):** Lưu trữ dữ liệu vĩnh viễn (persistent storage).
- **Thiết bị I/O:** Bàn phím, chuột, màn hình, kết nối mạng.

Thế giới số & Biểu diễn dữ liệu bằng Hệ nhị phân (Binary)

Tại sao máy tính sử dụng số nhị phân?

- Phần cứng vật lý cấu tạo từ hàng tỷ bóng bán dẫn (*transistors*).
- Hai trạng thái vật lý ổn định nhất:
  - **Bật (On) / Điện áp cao:** Đại diện cho số **1**.
  - **Tắt (Off) / Điện áp thấp:** Đại diện cho số **0**.
- **Bit (Binary Digit):** Đơn vị thông tin nhỏ nhất (0 hoặc 1).
- **Byte:** Nhóm 8 bits. 1 Byte biểu diễn được $2^8 = 256$ trạng thái (từ 0 đến 255).

Cơ chế biểu diễn số theo cơ số 2

So sánh hệ thập phân (Base-10) và nhị phân (Base-2):

Trọng số	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Bit	**1**	**0**	**1**	**1**

$$(1011)_2 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1$$
$$= 8 + 0 + 2 + 1 = \mathbf{11}_{10}$$

$\rightarrow$ Mọi chương trình, hình ảnh, video, âm thanh cuối cùng đều được ánh xạ về các dãy bit 0 và 1!

Cơ chế thực thi: Trình biên dịch (Compiler) vs. Thông dịch (Interpreter)

Trình biên dịch truyền thống (C/C++)

- Dịch toàn bộ mã nguồn (*Source code*) thành **Mã máy trực tiếp (Native Machine Code)**.
- **Ưu điểm:** Tốc độ thực thi cực nhanh, tối ưu phần cứng.
- **Nhược điểm chí tử:** **Phụ thuộc nền tảng!** File thực thi trên Intel x86 Windows không thể chạy trên ARM macOS hay Linux.

Trình thông dịch (Python, JavaScript)

- Đọc và diễn giải trực tiếp từng dòng lệnh tại thời điểm chạy.
- **Ưu điểm:** Linh hoạt, dễ thử nghiệm trực tiếp.
- **Nhược điểm:** Tốc độ thực thi chậm hơn nhiều so với mã đã biên dịch sẵn.

Câu hỏi đặt ra cho các nhà thiết kế ngôn ngữ Java (Năm 1995)

Làm sao vừa đạt được tốc độ thực thi cao, vừa có thể chạy trên **mọi nền tảng phần cứng** mà không cần biên dịch lại?

Kiến trúc đột phá của Java: Máy ảo JVM & Mã Bytecode

Triết lý: "Write Once, Run Anywhere" (WORA)

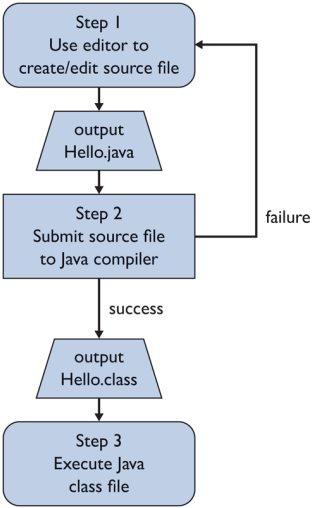
- Java kết hợp cả **Biên dịch** lẫn **Máy ảo**!
- Trình biên dịch Java (`javac`) không dịch ra mã máy thật của CPU.
- Nó dịch mã nguồn (`.java`) thành **Java Bytecode** (`.class`).
- Bytecode là tập lệnh chuẩn cho một cỗ máy lý thuyết gọi là **Java Virtual Machine (JVM)**.
- Mỗi hệ điều hành (Windows, Linux, macOS) có một bản JVM riêng biệt để thông dịch/JIT-compile Bytecode ra mã máy phần cứng.

Bộ công cụ Java (Phân biệt JDK, JRE, JVM)

- **JVM (Java Virtual Machine):** Môi trường thực thi Bytecode.
- **JRE (Java Runtime Environment):** Gồm JVM + Thư viện chuẩn (*Java Class Libraries*) để người dùng chạy ứng dụng.
- **JDK (Java Development Kit):** Dành cho lập trình viên! Chứa JRE + Trình biên dịch (`javac`) + Công cụ gỡ lỗi.

JDK = JRE + Tools ⊃
JRE = JVM + Libs

Quy trình phát triển chương trình Java: Edit → Compile → Run



Hình 1.1: Quá trình tạo và thực thi chương trình Java (Trích giáo trình Chapter 1, p.21)

3 Bước cốt lõi của mọi lập trình viên Java:

1. **Bước 1: Soạn thảo (Edit)**
   - Viết mã nguồn vào file văn bản thuần.
   - Bắt buộc đuôi file: `Hello.java`.
2. **Bước 2: Biên dịch (Compile)**
   - Lệnh: `javac Hello.java`
   - Kiểm tra lỗi cú pháp.
   - Tạo ra file bytecode: `Hello.class`.
3. **Bước 3: Thực thi (Run)**
   - Khởi động máy ảo JVM: `java Hello`
   - JVM nạp bytecode và chuyển cho CPU thực thi.
   - Kết quả xuất hiện trên màn hình!

1.2 Chương trình Java hoàn chỉnh đầu tiên

Listing 1: File: Hello.java

```java
// Chuong trinh Java dau tien cua lop IT26M
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}

```

Quy tắc vàng bắt buộc:

- Mọi mã nguồn Java đều phải nằm **bên trong một lớp (class)**.
- Tên file **bắt buộc phải trùng khớp 100%** với tên lớp công khai:

  Lớp `public class Hello` $\Longleftrightarrow$ File `Hello.java`
- Java phân biệt chữ hoa và chữ thường (*case-sensitive*).

Cấu trúc mẫu (Syntax Template)

Khai báo lớp chuẩn:

```
public class <TênLớp> {
... các phương thức ...
}
```

Cặp ngoặc nhọn { } :
Đóng gói phạm vi (*scope*) của khối lệnh. Mọi ngoặc mở { đều phải có ngoặc đóng } tương ứng.

Mổ xẻ phương thức `main` – Điểm bắt đầu của mọi ứng dụng

Dòng lệnh then chốt:

```
public static void main(String[] args) { ... }
```

Từ khóa	Ý nghĩa học thuật	Diễn giải dành cho sinh viên mới học
`public`	Quyền truy cập công khai	Bất kỳ đối tượng nào (bao gồm máy ảo JVM) cũng có thể gọi phương thức này từ bên ngoài.
`static`	Phương thức tĩnh cấp lớp	Có thể thực thi trực tiếp từ lớp mà không cần khởi tạo đối tượng (*instance*).
`void`	Kiểu dữ liệu trả về	Phương thức này thực hiện công việc và **không trả lại** giá trị nào.
`main`	Tên phương thức	Tên quy ước đặc biệt mà JVM tìm kiếm đầu tiên khi chạy chương trình.
`String[] args`	Mảng tham số dòng lệnh	Cho phép truyền các chuỗi đối số từ bên ngoài khi khởi động chương trình.

Lệnh xuất dữ liệu: `println` vs. `print`

System.out.println(...)

In nội dung ra console và **tự động xuống dòng mới** (newline).

```
1 System.out.println("Dong thu nhat");
2 System.out.println("Dong thu hai");
3
```

Kết quả trên màn hình:

```
Dong thu nhat
Dong thu hai
```

System.out.print(...)

In nội dung ra console và **giữ nguyên con trỏ** ở cuối dòng hiện tại.

```
1 System.out.print("Xin ");
2 System.out.print("chao ");
3 System.out.println("IT26M!");
4
```

Kết quả trên màn hình:

```
Xin chao IT26M!
```

Lưu ý quan trọng:

Lệnh `System.out.println();` (không tham số) dùng để tạo ra một dòng trống trên console. Mọi câu lệnh trong Java **bắt buộc kết thúc bằng dấu chấm phẩy ;**!

[CHECKPOINT 1] Kiểm tra nhanh lý thuyết Tiết 1

Câu hỏi 1: Lệnh nào hợp lệ trong Java?

A. `system.out.println("Hello");`
B. `System.Out.println("Hello");`
C. `System.out.println("Hello");`
D. `System.out.println(Hello);`

Câu hỏi 2: Vai trò của file có đuôi `.class`?

A. Chứa mã nguồn viết bằng tiếng Anh cho lập trình viên đọc.
B. Chứa mã Bytecode đã được biên dịch để máy ảo JVM thực thi.
C. Là file mã máy nhị phân chạy trực tiếp trên chip Intel.
D. Là file cấu hình môi trường của IDE.

Giải thích đáp án

Đáp án Câu 1: C
Java phân biệt hoa thường: `System` viết hoa chữ S, `out` và `println` viết thường. Chuỗi chữ phải nằm trong cặp nháy kép " ".

Đáp án Câu 2: B
Trình biên dịch `javac` sinh ra file `.class` chứa Bytecode độc lập nền tảng cho JVM.

[HANDS-ON LAB 1] Thực hành tại chỗ: Viết, biên dịch & chạy Java

Nhiệm vụ thực hành Tiết 1 (Thời gian: 15 phút – Tất cả sinh viên thực hiện)

Yêu cầu 1: Thao tác Command Line (Terminal / CMD / PowerShell)

1. Mở Terminal, gõ `javac -version` và `java -version` để kiểm tra JDK.
2. Dùng trình soạn thảo (Notepad / VS Code) tạo thư mục `Lab1` và file `WelcomeIT26M.java`.
3. Biên dịch bằng lệnh CLI: `javac WelcomeIT26M.java` → Dùng `dir` hoặc `ls` quan sát file `WelcomeIT26M.class` vừa xuất hiện!
4. Chạy chương trình: `java WelcomeIT26M`

Listing 2: Mã nguồn WelcomeIT26M.java

```
1  public class WelcomeIT26M {
2      public static void main(String[] args) {
3          System.out.println("Chao mung lop IT26M den voi
           Java!");
4          System.out.println("Viet mot lan, chay moi noi
           (WORA).");
5      }
6  }
```

Output hiển thị chính xác:

```
Chao mung lop IT26M den voi Java!
Viet mot lan, chay moi noi (WORA).
```

TIẾT 2

CHUỖI KÝ TỰ, KÝ TỰ THOÁT (ESCAPE SEQUENCES)

& KỸ NĂNG CHẨN ĐOÁN, GỠ LỖI LẬP TRÌNH (DEBUGGING)

Thời lượng: 50 phút | 35’ Lý thuyết & Thảo luận + 15’ Hands-on Lab 2

Chuỗi ký tự (String Literals) & Thách thức khi xuất ký tự đặc biệt

Chuỗi ký tự (String Literal)

- Là một chuỗi các ký tự nằm trong cặp **dấu ngoặc kép** (`"..."`).
- Ví dụ: `"Xin chao"`, `"IT26M"`, `""` (chuỗi rỗng).
- **Cấm:** Không được ngắt một chuỗi qua nhiều dòng mà không đóng ngoặc kép!

Vấn đề hóc búa nảy sinh:

Nếu muốn in ra màn hình chính xác câu sau:

```
Giao vien noi: "Java rat thu vi!"
```

Nếu viết:

```
System.out.println("Giao vien noi: "Java rat thu vi!"");
```

→ **Trình biên dịch sẽ báo lỗi ngay!** Vì nó nghĩ chuỗi kết thúc ở dấu ngoặc kép thứ hai.

Giải pháp trong mọi ngôn ngữ lập trình: Sử dụng **Ký tự thoát (Escape Sequences)**!

Bảng ký tự thoát chuẩn trong Java (Escape Sequences)

Quy tắc: Dấu gạch chéo ngược (\) kết hợp với ký tự đi liền sau:

Ký tự thoát	Tên gọi	Ý nghĩa & Hành vi thực hiện
`\t`	Tab	Thêm khoảng thụt lề tab (căn chỉnh cột dữ liệu đẹp mắt).
`\n`	Newline	Xuống dòng mới ngay tại vị trí xuất hiện.
`\"`	Double quote	In dấu ngoặc kép " mà không làm đóng chuỗi.
`\\`	Backslash	In một dấu gạch chéo ngược \.

Listing 3: Ví dụ sử dụng Escape Sequences

```
1 System.out.println("Cot 1\tCot 2\tCot 3");
2 System.out.println("Dong 1\nDong 2\nDong 3");
3 System.out.println("Dau \"nhay kep\" va gach \\");
4
```

Kết quả in ra màn hình:

```
Cot 1   Cot 2   Cot 3
Dong 1
Dong 2
Dong 3
Dau "nhay kep" va gach \
```

Định danh (Identifiers) & Từ khóa chuẩn Java

Định danh (Identifiers)

Tên do lập trình viên đặt cho lớp, phương thức, biến...

- **Ký tự hợp lệ:** Các chữ cái (`a-z, A-Z`), chữ số (`0-9`), dấu gạch dưới (_), dấu đô la ($).
- **Quy tắc 1: Không được bắt đầu bằng chữ số!** (Hợp lệ: `diem1`, không hợp lệ: `1diem`).
- **Quy tắc 2:** Không chứa dấu cách hoặc ký tự đặc biệt (`@, #, %, -, +`).
- **Quy tắc 3: Không trùng với Từ khóa Java!**

Từ khóa Java (Keywords – Dành riêng)

Java có 50+ từ khóa đặc biệt đã được định nghĩa sẵn cú pháp:

`public`	`class`	`static`	`void`
`int`	`double`	`boolean`	`char`
`if`	`else`	`for`	`while`
`return`	`new`	`true`	`false`

Tuyệt đối không dùng các từ trên làm tên Class hay tên hàm!

Phong cách lập trình chuẩn đại học: Quy ước CamelCase & Comments

Quy ước đặt tên CamelCase

- **Tên Lớp (UpperCamelCase):**
  Viết hoa chữ cái đầu của mỗi từ.
  Ví dụ: `Hello`, `DrawFigures3`, `StudentManager`.
- **Tên Phương thức & Biến (lowerCamelCase):**
  Chữ cái đầu viết thường, viết hoa các từ tiếp theo.
  Ví dụ: `main`, `drawDiamond`, `calculateTotalScore`.

3 Loại chú thích (Comments) trong Java

- **Chú thích 1 dòng:** Bắt đầu bằng `//`
  `// Tinh tong diem cua sinh vien`
- **Chú thích nhiều dòng:** Cặp `/* ... */`
  ```
  /* Tac gia: IT26M
     Ngay tao: 2026 */
  ```
- **Chú thích tài liệu (Javadoc):** Cặp `/** ... */`
  Dùng để sinh tài liệu API tự động.

*"Mã nguồn viết ra cho người đọc hiểu trước, máy tính hiểu sau." – Thụt lề (**Indentation**) nhất quán 4 khoảng trắng!*

1.3 Phân loại lỗi lập trình: 3 nhóm lỗi kinh điển

Mọi lập trình viên từ sinh viên năm nhất đến chuyên gia thâm niên đều phải đối mặt với lỗi!

1. Lỗi Cú pháp (Syntax Errors)

- Xảy ra trong giai đoạn **Biên dịch (Compile-time)**.
- Vi phạm ngữ pháp ngôn ngữ: thiếu dấu `;`, sai chính tả từ khóa, quên đóng ngoặc `}`.
- **Đặc điểm:** Trình biên dịch `javac` phát hiện và từ chối sinh file `.class`.

2. Lỗi Thực thi (Runtime Errors)

- Biên dịch thành công, xảy ra khi **Đang chạy (Run-time)**.
- Chương trình yêu cầu CPU thực hiện phép toán bất khả thi (chia cho 0, truy cập mảng rỗng).
- **Đặc điểm:** Chương trình bị sập (*crash*) và tung ra ngoại lệ (**Exception**).

3. Lỗi Ngữ nghĩa (Logic / Intent Errors)

- Chương trình biên dịch tốt, chạy không bị sập.
- **Nhưng kết quả sai** so với yêu cầu ban đầu!
- Ví dụ: Cần tính tiền thuế mà viết nhầm dấu `+` thành dấu `-`.
- **Đặc điểm:** Nguy hiểm nhất vì máy tính không thể tự phát hiện!

Kỹ năng đọc thông báo lỗi từ Trình biên dịch (`javac`)

Cấu trúc một thông báo lỗi điển hình:

```
Hello.java:3: error: ';' expected
        System.out.println("Hello, world!")
                                           ^
1 error
```

Các thành phần của thông báo lỗi

1. **Tên file bị lỗi:** `Hello.java`
2. **Dòng bị lỗi:** Dòng số `3`
3. **Loại lỗi:** `';' expected` (Thiếu dấu chấm phẩy)
4. **Vị trí con trỏ (^):** Chỉ ra điểm chính xác mà compiler phát hiện bất thường.

Kinh nghiệm vàng khi Debug:

- **Quy tắc 1:** Luôn nhìn vào **lỗi đầu tiên** trong danh sách lỗi! Một lỗi ở dòng 3 có thể gây ra hiệu ứng domino tạo 10 lỗi giả phía sau.
- **Quy tắc 2:** Nếu dòng báo lỗi trông hoàn toàn đúng, hãy kiểm tra **dòng ngay phía trước nó**.

[CHECKPOINT 2] Thử tài thám tử chẩn đoán lỗi

Đoạn chương trình sau chứa 3 lỗi cú pháp. Bạn hãy tìm ra chúng?

```
1 public MyProgram {
2     public static void Main(String[] args) {
3         System.out.println("Lop IT26M)
4     }
5 }
6
```

Chẩn đoán và phân tích chi tiết:

1. **Lỗi dòng 1:** Thiếu từ khóa **`class`** giữa `public` và `MyProgram` → `public class MyProgram`.
2. **Lỗi dòng 2:** Tên hàm `main` bị viết hoa chữ M → Java phân biệt hoa thường, phải là **`main`**.
3. **Lỗi dòng 3:** Chuỗi ký tự thiếu dấu ngoặc kép đóng `"` và dấu chấm phẩy `;` kết thúc lệnh → `System.out.println("Lop IT26M");`.

[HANDS-ON LAB 2] Thực hành Debug & Chuỗi ký tự thoát

Thử thách thực hành Tiết 2 (Thời gian: 15 phút – Trực tiếp trên máy tính)

Bài toán: Viết chương trình Java hoàn chỉnh tên là `EscapeChallenge.java` để in ra màn hình chính xác đoạn văn bản sau (chú ý từng dấu ngoặc kép, gạch chéo và tab):

Kết quả mong đợi trên Console:

```
Dac ta he thong:
    1. Thu muc: C:\Java\IT26M
    2. Khau hieu: "Hoc la phai hanh!"
    3. Tam giac: /\  /\
```

Gợi ý kỹ thuật:

- Dấu gạch chéo ngược trong đường dẫn: Dùng `\\`.
- Dấu ngoặc kép trong khẩu hiệu: Dùng `\"`.
- Thụt lề đầu dòng: Dùng `\t`.
- Hãy thử cố tình gõ thiếu dấu `\` để xem trình biên dịch báo lỗi gì!

TIẾT 3

PHÂN RÃ THỦ TỤC (PROCEDURAL DECOMPOSITION)

PHƯƠNG THỨC TĨNH & TIẾN HÓA CASE STUDY DRAWFIGURES

Thời lượng: 50 phút | 30’ Lý thuyết thiết kế + 20’ Mini Project Lab 3

1.4 Vấn đề của bài toán lớn & Tư duy Phân rã Top-down

Cạm bẫy của chương trình nguyên khối (Monolithic)

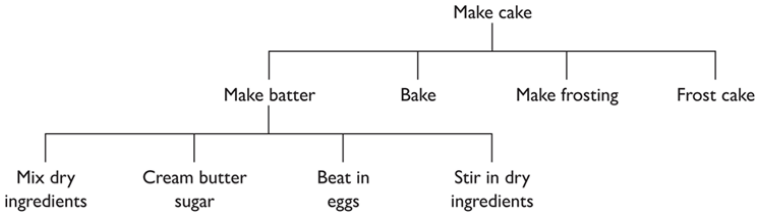
- Đặt tất cả hàng trăm dòng lệnh vào phương thức `main`.
- Mã nguồn cực kỳ khó đọc, khó hiểu (*spaghetti code*).
- **Trùng lặp mã nguồn (Code Duplication):** Cùng một khối lệnh phải gõ đi gõ lại nhiều lần!
- Rất khó sửa lỗi và không thể tái sử dụng.

Nguyên lý Phân rã Top-down (Divide and Conquer)

- Chia bài toán lớn phức tạp thành các **bài toán con (subtasks)**.
- Tiếp tục chia nhỏ bài toán con cho đến khi đạt được các thao tác đơn giản nhất (*primitives*).
- Giải quyết từng bài toán con bằng các **phương thức độc lập**.

Nguyên lý cốt lõi trong kỹ nghệ phần mềm: DRY – Don't Repeat Yourself!

Ví dụ thực tế: Phân rã tác vụ “Làm bánh” (Make Cake)



Hình 1.2: Sơ đồ cấu trúc phân rã tác vụ làm bánh (Trích giáo trình Chapter 1, Trang 66)

Đặc điểm của Sơ đồ Cấu trúc (Structure Diagram)

- **Cấp cao nhất (Top level):** Nhiệm vụ tổng thể (*Make cake*).
- **Cấp 1:** Các giai đoạn chính (*Make batter*, *Bake*, *Make frosting*, *Frost cake*).
- **Cấp 2:** Thao tác cụ thể (*Mix dry ingredients*, *Cream butter and sugar*...).

Ý nghĩa sư phạm trong lập trình

- Giúp người lập trình có cái nhìn tổng quan trước khi viết từng dòng code.
- Trong Java, mỗi hộp trong sơ đồ cấu trúc sẽ tương ứng với một **phương thức tĩnh (static method)**.

Phương thức tĩnh (Static Methods): Cú pháp Khai báo & Lời gọi

Cú pháp Khai báo (Declaring a Method)

Định nghĩa tên và hành vi của phương thức:

```
public static void <tenPhuongThuc>() {
    <cac cau lenh thuc thi>;
}

```

Ví dụ:

```
public static void drawBox() {
    System.out.println("+------+");
    System.out.println("|      |");
    System.out.println("|      |");
    System.out.println("+------+");
}

```

Cú pháp Lời gọi (Calling a Method)

Ra lệnh cho máy tính thực thi phương thức:

```
<tenPhuongThuc>();

```

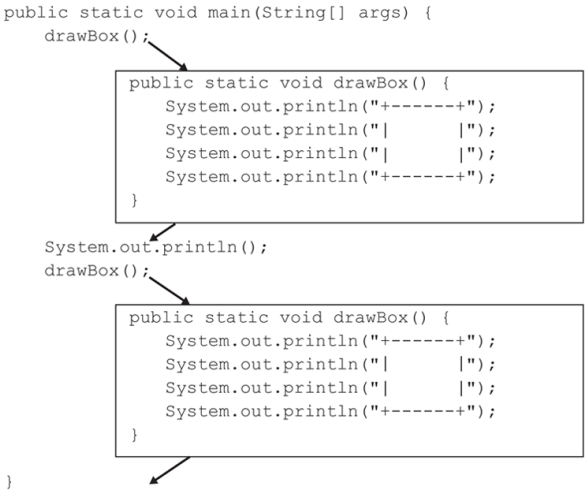
Ví dụ trong `main`:

```
public static void main(String[] args) {
    drawBox(); // Goi lan 1
    System.out.println();
    drawBox(); // Goi lan 2 (Tai su dung!)
}

```

Lưu ý

Luồng điều khiển khi gọi phương thức (Flow of Control)

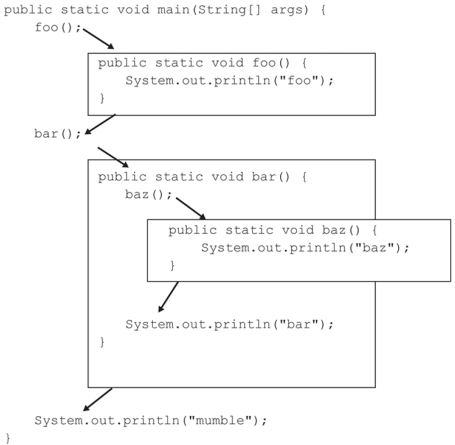


Sơ đồ luồng điều khiển chương trình DrawBoxes (Trích giáo trình Chapter 1, Trang 76)

Quy tắc chuyển giao điều khiển:

1. Chương trình **luôn bắt đầu** tại dòng đầu tiên của hàm `main`.
2. Khi gặp lời gọi hàm (`drawBox();`):
   - Máy tính **tạm dừng** thực thi tại `main`.
   - Nhảy tới thân phương thức `drawBox` và thực thi tuần tự các lệnh trong đó.
3. Khi phương thức chạy xong dấu ngoặc đóng `}`:
   - Quyền điều khiển quay trở lại (**return**) đúng vị trí vừa rời đi ở `main`.
   - Tiếp tục thực thi câu lệnh kế tiếp!

Luồng điều khiển đa cấp & Khái niệm Ngăn xếp gọi hàm (Call Stack)



Sự mở rộng luồng điều khiển lồng nhau foo → bar → baz (Trang 82)

Cơ chế hoạt động của Call Stack

- Một phương thức có thể gọi một phương thức khác (`bar()` gọi `baz()`).
- Máy tính ghi nhớ thứ tự quay về bằng cấu trúc **Ngăn xếp (Call Stack - LIFO: Vào sau ra trước)**.
- Khi `baz()` chạy xong → quay về `bar()` → `bar()` chạy xong → quay về `main`.

Lợi ích kép của Phân rã thủ tục:

1. **Structure (Cấu trúc):** Biến chương trình dài phức tạp thành các module ngắn gọn, dễ hiểu.
2. **Eliminate Redundancy (Xóa bỏ trùng lặp):** Viết 1 lần, gọi thực thi vô số lần!

1.5 Case Study: Bài toán vẽ hình ASCII nghệ thuật (DrawFigures)

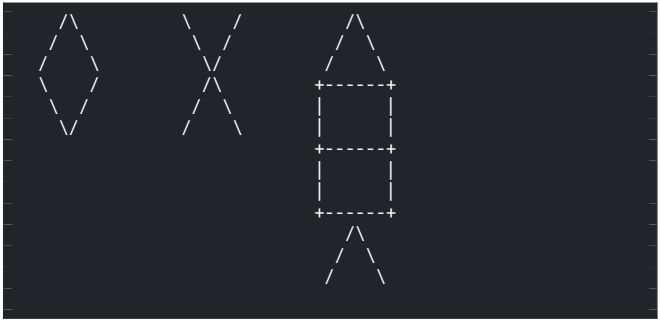
Yêu cầu bài toán (Output mong đợi)

Viết chương trình in ra 3 hình đồ họa ký tự ASCII:

1. **Hình Kim cương (Diamond)**
2. **Hình Chữ X (X)**
3. **Hình Tên lửa vũ trụ (Rocket)**

Các hình cách nhau một dòng trống.

Hình ảnh ASCII thực tế xuất ra Console:



Phiên bản 1 (DrawFigures1): Tiếp cận Ngây thơ (Naïve Approach)

Listing 4: DrawFigures1.java (Trích đoạn)

```java
public class DrawFigures1 {
    public static void main(String[] args) {
        // Ve kim cuong (Diamond)
        System.out.println("   /\\");
        System.out.println("  /  \\");
        System.out.println(" /    \\");
        System.out.println(" \\    /");
        System.out.println("  \\  /");
        System.out.println("   \\/");
        System.out.println();
        // Ve chu X
        System.out.println(" \\    /");
        System.out.println("  \\  /");
        System.out.println("   \\/");
        System.out.println("   /\\");
        // ... tiep tuc hang chuc dong lenh!
    }
}

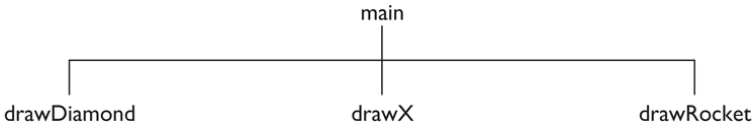
```

Đánh giá chất lượng thiết kế:

- **Số dòng lệnh:** Hơn 30 dòng lệnh `println` dồn hết vào hàm `main`.
- **Trùng lặp trầm trọng:** Cụm nón trên (*cone*) lặp lại 3 lần. Cụm nón dưới (*v*) lặp lại 2 lần. Cụm hình hộp lặp lại 2 lần.
- **Hậu quả bảo trì:** Nếu muốn mở rộng hình rộng hơn 1 dấu cách, ta phải đi sửa thủ công ở hàng chục vị trí khác nhau!

→ **Cần áp dụng Phân rã thủ tục cấp 1!**

Phiên bản 2 (DrawFigures2): Phân rã Cấp 1 theo Khối hình



Hình 1.3: Sơ đồ cấu trúc phân rã cấp 1 của DrawFigures2 (Trích giáo trình Chapter 1, Trang 92)

Cải tiến ở Phiên bản 2:

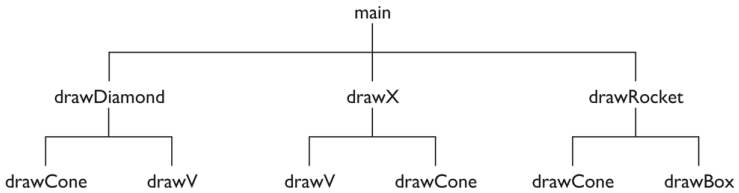
- Tách `main` thành 3 phương thức con:
  - `drawDiamond()`
  - `drawX()`
  - `drawRocket()`
- Hàm `main` bây giờ cực kỳ ngắn gọn và sáng sủa:

```
drawDiamond(); drawX(); drawRocket();
```

Hạn chế vẫn còn tồn tại:

- Bên trong thân của `drawDiamond`, `drawX`, và `drawRocket`, các câu lệnh in nón xuôi (`/\`) và nón ngược (`\/`) **vẫn bị lặp lại**!
- **Giải pháp triệt để:** Tiếp tục phân rã xuống cấp thứ 2 thành các hình khối cơ sở (*primitives*)!

Phiên bản 3 (DrawFigures3): Phân rã Hoàn chỉnh 2 Cấp (Triệt tiêu trùng lặp)



Hình 1.4: Sơ đồ cấu trúc phân rã hoàn chỉnh 2 cấp của DrawFigures3 (Trích giáo trình Chapter 1, Trang 96)

Cấu trúc 3 tầng chuẩn kiến trúc phần mềm:

- **Tầng 1 (Điều phối):** Phương thức `main()` nắm giữ bức tranh toàn cảnh cấp cao nhất.
- **Tầng 2 (Khối hình hoàn chỉnh):** `drawDiamond()`, `drawX()`, `drawRocket()`.
- **Tầng 3 (Các khối nguyên thủy tái sử dụng):** `drawCone()` (Nón trên), `drawV()` (Nón dưới), `drawBox()` (Thân hộp).

Mã nguồn hoàn chỉnh đạt chuẩn: `DrawFigures3.java`

Listing 5: Tầng 1 & Tầng 2

```java
public class DrawFigures3 {
    public static void main(String[] args) {
        drawDiamond();
        System.out.println();
        drawX();
        System.out.println();
        drawRocket();
    }
    public static void drawDiamond() {
        drawCone();
        drawV();
    }
    public static void drawX() {
        drawV();
        drawCone();
    }
    public static void drawRocket() {
        drawCone();
        drawBox();
        drawBox();
        drawCone();
    }
```

Listing 6: Tầng 3 (Primitives Reusable)

```java
    public static void drawCone() {
        System.out.println("   /\\");
        System.out.println("  /  \\");
        System.out.println(" /    \\");
    }
    public static void drawV() {
        System.out.println(" \\    /");
        System.out.println("  \\  /");
        System.out.println("   \\/");
    }
    public static void drawBox() {
        System.out.println("+------+");
        System.out.println("|      |");
        System.out.println("|      |");
        System.out.println("+------+");
    }
}

```

So sánh & Đánh giá 3 thế hệ thiết kế

Tiêu chí đánh giá	DrawFigures1 (Naïve)	DrawFigures2 (Cấp 1)	DrawFigures3 (Cấp 2)
Cấu trúc mã nguồn	Rối rắm, nguyên khối 1 hàm duy nhất.	Tách biệt từng hình lớn, dễ theo dõi hơn.	Phân tầng rõ rệt: Điều phối - Lắp ráp - Nguyên tử.
Mức độ trùng lặp	Rất cao (*Heavy redundancy*).	Vẫn lặp code vẽ nón và vẽ hộp.	**Triệt tiêu 100%** trùng lặp code in hình.
Khả năng tái sử dụng	0% (phải copy-paste dòng lệnh).	Chỉ tái sử dụng được cả hình kim cương hay chữ X.	**Tái sử dụng tối đa** mọi khối nguyên thủy.
Khả năng bảo trì	Sửa đổi 1 chi tiết phải sửa hàng chục chỗ.	Vẫn phải sửa nhiều nơi.	**Chỉ cần sửa 1 chỗ duy nhất** tại khối nguyên thủy!

[CHECKPOINT 3] Dự đoán luồng thực thi hàm lồng nhau

```java
public class TraceTest {
    public static void main(String[] args) {
        first();
        second();
    }
    public static void first() {
        System.out.println("1");
    }
    public static void second() {
        System.out.println("2");
        first();
        System.out.println("3");
    }
}

```

Thứ tự in trên màn hình là gì?

A. 1, 2, 3
B. 1, 2, 1, 3
C. 1, 2, 3, 1
D. 2, 1, 3

Giải thích:

1. `main` gọi `first()` → in **1**.
2. `main` gọi `second()`:
- In **2**.
- Gọi `first()` → in **1**.
- In **3**.
→ **Đáp án chính xác: B (1, 2, 1, 3)**

[HANDS-ON LAB 3] Mini-Project: Thiết kế chương trình Rocket/FightSong

Mini Project Tiết 3 (Thời gian: 20 phút – Đánh giá sản phẩm tại lớp)

Đề bài (Chọn 1 trong 2):

- **Lựa chọn A (Đồ họa ASCII):** Viết `SpaceRocket.java` vẽ một trạm không gian gồm 2 tên lửa đứng cạnh nhau, mỗi tên lửa gồm mũi nhọn, thân chứa biểu tượng `IT26M`, khoang động cơ, và ngọn lửa đáy. Yêu cầu **triệt tiêu 100% trùng lặp**.
- **Lựa chọn B (Bài hát cổ động - Exercise 12, p.125):** Viết chương trình `FightSong.java` in lời bài hát có các điệp khúc lặp lại bằng cách phân rã thành ít nhất 2 phương thức tĩnh phụ ngoài `main`.

Tiêu chí chấm điểm Lab:

1. Biên dịch và chạy hoàn hảo (`0 errors, 0 warnings`).
2. Cấu trúc phân rã rõ ràng (Tối thiểu 2 cấp phân rã).
3. Tuân thủ chuẩn đặt tên CamelCase và chú thích code.

Cấu trúc mẫu SpaceRocket:

- `drawNoseCone()`
- `drawBodySegment()`
- `drawThruster()`
- `drawFlame()`

Tổng kết toàn bộ kiến thức Chương 1 (Checklist Chuẩn đầu ra)

Mục tiêu CLO	Kiến thức cốt lõi đã làm chủ trong 3 tiết	Trạng thái
CLO 1.1	Hiểu phần cứng, nhị phân, vai trò của Bytecode và máy ảo JVM (WORA).	✓
CLO 1.2	Viết đúng cấu trúc class, hàm `main`, phân biệt `print`/`println`, dùng thành thạo ký tự thoát (`\n, \t, \", \\`).	✓
CLO 1.3	Nhận diện và phân biệt 3 loại lỗi: Cú pháp (Syntax), Thực thi (Runtime), Ngữ nghĩa (Logic); đọc hiểu thông báo lỗi compiler.	✓
CLO 1.4	Nắm vững tư duy phân rã Top-down; khai báo và gọi phương thức tĩnh; hiểu luồng điều khiển và ngăn xếp gọi hàm.	✓
CLO 1.5	Viết mã nguồn có cấu trúc module chuẩn, áp dụng nguyên lý DRY, biên dịch và chạy độc lập trên CLI và IDE.	✓

Bảng tra cứu nhanh cú pháp cốt lõi (Java Quick Cheatsheet)

Cấu trúc chương trình & Xuất màn hình

Khung chương trình:

```
public class TênLớp {
public static void main(String[] args) {
System.out.println("Nội dung");
}
}
```

Ký tự thoát:

`\n` (Xuống dòng) `\t` (Tab)
`\"` (Dấu ngoặc kép) `\\` (Gạch chéo ngược)

Phương thức tĩnh & Phân rã

Khai báo hàm:

```
public static void tênHàm() {
... các câu lệnh ...
}
```

Lời gọi hàm:

```
tênHàm();
```

Quy tắc vàng:

- Hàm không được lồng trong hàm khác.
- Đặt tên theo chuẩn `lowerCamelCase`.
- Triệt tiêu trùng lặp code (DRY).

Nhiệm vụ tự học & Bài tập về nhà cho lớp IT26M

1. Bài tập bắt buộc (Nộp trên hệ thống LMS)

- Đọc lại toàn bộ Chương 1 giáo trình (Trang 1 – 102).
- Làm các câu hỏi **Self-Check**:
  - Câu 6, 8, 10, 13 (Cú pháp & Escape Seq).
  - Câu 18, 19 (Tìm lỗi chương trình).
  - Câu 24, 26, 27 (Luồng điều khiển).
- Lập trình **Exercise 7 (Mantra)** và **Exercise 11 (TwoRockets)** trang 122–125.

2. Bài tập thử thách (Cộng điểm chuyên cần)

- **Programming Project 1 (MISSISSIPPI):** In chữ MISSISSIPPI dạng khối lớn (Block letters) với phân rã tối ưu (p.130).
- **Programming Project 3:** Bài hát dân gian tích lũy điệp khúc có cấu trúc phân tầng (p.131).

Chuẩn bị cho Buổi học kế tiếp:

Đọc trước **Chương 2: Primitive Data and Definite Loops** (Biến, Kiểu số nguyên/thực, Biểu thức toán học, và Vòng lặp `for`).

CẢM ƠN CÁC BẠN SINH VIÊN LỚP IT26M!

HỎI ĐÁP & THẢO LUẬN (Q&A)

Chúc các bạn có những giờ học lập trình Java say mê và hiệu quả!

Khoa Công nghệ Thông tin – Đại học UDA